

Shor's Factoring Algorithm

The one that started the funding. Period finding via the quantum Fourier transform, and an honest look at the qubit count RSA-2048 needs.

EXAMPLE NOTES

Written by the QuantaVerse team to show what a module's notes can look like: each lesson's theory in brief, every formula typeset and numbered, and the key ones gathered on a formula sheet at the end. Professors can upload their own notes for this module on its page, and students will see them alongside these.

8.1	Why factoring matters	2
	The assumption a great deal of the internet is resting on.	
8.2	Factoring as period finding	3
	The reduction that turns a number-theory problem into a wave problem.	
8.3	Modular exponentiation	4
	The part of the circuit that does the real work, and costs the most.	
8.4	The quantum Fourier transform	5
	The same transform as the classical one, applied to amplitudes.	
8.5	Reading a period	6
	What the measurement after the QFT actually gives you.	
8.6	Continued fractions	7
	The classical trick that recovers r from an approximate fraction.	
8.7	The whole algorithm	8
	Every step in order, and how little of it is quantum.	
8.8	Error budgets	9
	Why the famous demonstrations factor 15, and why that is not a scandal.	
8.9	What this means for cryptography	10
	What breaks, what does not, and why the migration started already.	
Σ	Formula sheet	11
	Every key formula of the module, gathered at the end.	

CONVENTIONS

N is the number to factor, with $n = \lceil \log_2 N \rceil$ bits. a is a random base with $\gcd(a, N) = 1$, and r is its order modulo N . The counting register has t qubits and $M = 2^t$ basis states. $a \bmod N$ is the remainder, and $\equiv \pmod{N}$ is congruence.

LESSON 8.1

Why factoring matters

KEY IDEA

Multiplying two large primes is easy; recovering them from the product is believed to be classically hard. RSA's security rests on that gap, and Shor's algorithm closes it.

$$N = pq, \quad \varphi(N) = (p-1)(q-1), \quad ed \equiv 1 \pmod{\varphi(N)} \quad (8.1.1)$$

an RSA key: public (N, e) , private d

$$c = m^e \bmod N, \quad m = c^d \bmod N \quad (8.1.2)$$

encryption and decryption

Anyone who can factor N can compute $\varphi(N)$, then d from e , and decrypt everything. The best known classical method, the general number field sieve, is sub-exponential: faster than trying divisors, still far out of reach for 2048-bit moduli.

$$L_N = \exp\left(\left(\frac{64}{9}\right)^{1/3} (\ln N)^{1/3} (\ln \ln N)^{2/3}\right) \quad (8.1.3)$$

the number field sieve's running time, to leading order

$$\text{Shor: } O(n^3) \text{ gates for an } n\text{-bit } N \quad (8.1.4)$$

polynomial, with schoolbook arithmetic: faster multiplication lowers it further

METHOD	TIME FOR n -BIT N	CHARACTER
trial division	$\approx 2^{n/2}$	exponential
number field sieve	L_N above	sub-exponential
Shor	$O(n^3)$ gates	polynomial

WORKED EXAMPLE · RSA IN MINIATURE

$p = 5, q = 11$: $N = 55$ and $\varphi(N) = 40$. Take $e = 3$; then $d = 27$, since $3 \cdot 27 = 81 = 2 \cdot 40 + 1$.

Encrypt $m = 2$: $c = 2^3 \bmod 55 = 8$. Decrypt: $8^{27} \bmod 55 = 2$. Factoring 55 is trivial, which is the whole point: at 2048 bits it is not.

LESSON 8.2

Factoring as period finding

KEY IDEA

Factoring reduces to finding the order r of a random number a modulo N . The sequence $a^x \bmod N$ repeats with period r , and a quantum computer finds periods efficiently.

$$r = \min \{ r > 0 : a^r \equiv 1 \pmod{N} \} \quad (8.2.1)$$

the order of a modulo N , defined when $\gcd(a, N) = 1$

If r is even, $a^r - 1 = (a^{r/2} - 1)(a^{r/2} + 1)$ is a multiple of N . Unless $a^{r/2} \equiv -1$, neither bracket is itself a multiple of N , so each shares a proper factor with it, and a gcd extracts it.

$$\gcd(a^{r/2} - 1, N) \quad \text{and} \quad \gcd(a^{r/2} + 1, N) \quad (8.2.2)$$

the factors, when r is even and $a^{r/2} \not\equiv -1 \pmod{N}$

$$P(r \text{ even and } a^{r/2} \not\equiv -1) \geq \frac{1}{2} \quad (8.2.3)$$

for odd N with at least two distinct prime factors: a few random tries suffice

Everything in this reduction is classical and fast, including the gcds, by Euclid's algorithm. Only finding r is hard, and that is the part handed to the quantum computer.

a	POWERS OF $a \bmod 15$	r	$a^{r/2} \bmod 15$	FACTORS FOUND
2	2, 4, 8, 1	4	4	$\gcd(3, 15) = 3$, $\gcd(5, 15) = 5$
7	7, 4, 13, 1	4	4	3 and 5
11	11, 1	2	11	$\gcd(10, 15) = 5$, $\gcd(12, 15) = 3$
14	14, 1	2	$14 \equiv -1$	none: pick another a

WORKED EXAMPLE · $N = 15$, $A = 7$

$7^1 \equiv 7$, $7^2 \equiv 49 \equiv 4$, $7^3 \equiv 28 \equiv 13$, $7^4 \equiv 91 \equiv 1$, so $r = 4$.

$a^{r/2} = 7^2 \equiv 4$, which is not $-1 \equiv 14$. So $\gcd(3, 15) = 3$ and $\gcd(5, 15) = 5$: $15 = 3 \times 5$.

LESSON 8.3

Modular exponentiation

KEY IDEA

The circuit computes $a^x \bmod N$ for every x in a superposition at once, by controlled modular multiplications. It is ordinary arithmetic made reversible, and it dominates the cost.

$$U_a|y\rangle = |ay \bmod N\rangle \quad (0 \leq y < N) \quad (8.3.1)$$

modular multiplication: a permutation, so a unitary, because $\gcd(a, N) = 1$

$$|x\rangle|1\rangle \mapsto |x\rangle|a^x \bmod N\rangle, \quad a^x = \prod_{j=0}^{t-1} (a^{2^j})^{x_j} \quad (8.3.2)$$

controlled powers: one controlled $U_{a^{2^j}}$ per bit x_j of the counting register

The powers $a^{2^j} \bmod N$ are computed classically in advance by repeated squaring, so the circuit only ever multiplies by known constants. Each multiplication is built from reversible adders, which is where most qubits and most gates go.

The key to why this helps lies in the eigenvectors of U_a . They are uniform superpositions over the cycle $1, a, a^2, \dots$, with phases that wind s times around, and their eigenvalues carry s/r .

$$|u_s\rangle = \frac{1}{\sqrt{r}} \sum_{k=0}^{r-1} e^{-2\pi i s k / r} |a^k \bmod N\rangle, \quad U_a|u_s\rangle = e^{2\pi i s / r} |u_s\rangle \quad (8.3.3)$$

eigenstates of modular multiplication

$$\frac{1}{\sqrt{r}} \sum_{s=0}^{r-1} |u_s\rangle = |1\rangle \quad (8.3.4)$$

the easy input $|1\rangle$ is an equal mix of all of them

So starting the work register in $|1\rangle$ is starting in a superposition of eigenstates, each carrying a phase s/r . Phase estimation (lesson 8.4) reads one such s/r at random, which is exactly what lesson 8.6 needs.

REGISTER	QUBITS	ROLE
counting	$t \approx 2n$	holds x , later the phase estimate
work	n	holds $a^x \bmod N$
ancillas	$O(n)$	scratch space for the reversible adders

WORKED EXAMPLE · THE CYCLE FOR $a = 7, N = 15$

Multiplying by 7 cycles $1 \rightarrow 7 \rightarrow 4 \rightarrow 13 \rightarrow 1$, so $r = 4$.

The eigenstate with $s = 1$ is $\frac{1}{2}(|1\rangle - i|7\rangle - |4\rangle + i|13\rangle)$. Multiplying each term by 7 shifts every label one step along the cycle, and that is the same as multiplying the whole state by $e^{2\pi i / 4} = i$.

LESSON 8.4

The quantum Fourier transform

KEY IDEA

The QFT is the discrete Fourier transform acting on the 2^t amplitudes of a register. It needs only about $t^2/2$ gates, but its output can only be sampled, not read in full.

$$\text{QFT}|x\rangle = \frac{1}{\sqrt{M}} \sum_{y=0}^{M-1} e^{2\pi i xy/M} |y\rangle, \quad M = 2^t \quad (8.4.1)$$

the quantum Fourier transform

The circuit is a Hadamard on each qubit followed by controlled phase rotations from the less significant qubits, then a reversal of qubit order.

$$R_k = \begin{pmatrix} 1 & 0 \\ 0 & e^{2\pi i/2^k} \end{pmatrix}, \quad R_1 = Z, \quad R_2 = S, \quad R_3 = T \quad (8.4.2)$$

the controlled rotations

$$\# \text{gates} = \underbrace{t}_H + \underbrace{\frac{t(t-1)}{2}}_{\text{controlled } R_k} + \underbrace{\lfloor t/2 \rfloor}_{\text{swaps}} \quad (8.4.3)$$

circuit size

TRANSFORM	OPERATIONS ON $M = 2^t$ VALUES	OUTPUT
classical FFT	$O(t 2^t)$	every coefficient, exactly
QFT	$O(t^2)$ gates	one sample from the spectrum

In Shor's algorithm the QFT is used in reverse, as the last step of *phase estimation*. A register holding a phase gradient $e^{2\pi i \varphi x}$ is transformed into a state concentrated on the t -bit number closest to $M\varphi$.

$$\frac{1}{\sqrt{M}} \sum_{x=0}^{M-1} e^{2\pi i \varphi x} |x\rangle \xrightarrow{\text{QFT}^\dagger} \approx |\widetilde{M\varphi}\rangle \quad (8.4.4)$$

phase estimation: the phase, written out as a binary number

The controlled powers of lesson 8.3 create exactly that gradient, with $\varphi = s/r$, because an eigenstate kicks its phase back onto every control qubit, just as the oracle did in lesson 6.3.

Rotations R_k with large k change the state very little. Dropping those beyond $k \approx \log_2 t + O(1)$ gives the *approximate QFT*, with a negligible loss of accuracy and about $t \log t$ gates.

WORKED EXAMPLE · QFT OF A BASIS STATE, $t = 2$

$\text{QFT}|1\rangle = \frac{1}{2}(|0\rangle + i|1\rangle - |2\rangle - i|3\rangle)$: the phases $e^{2\pi i y/4} = 1, i, -1, -i$ wind once around the circle.

$|2\rangle$ would wind twice, giving $1, -1, 1, -1$. The winding number is the input, which is why $\text{QFT}^\dagger$ can read it back.

LESSON 8.5

Reading a period

KEY IDEA

The measurement does not return r . It returns a number y close to kM/r for a random k , so y/M approximates the fraction k/r .

After the controlled powers, the counting register holds a superposition that repeats with period r , whether or not the work register is ever measured. The inverse QFT turns that periodicity into sharp peaks.

$$P(y) \approx \frac{1}{r} \text{ for } y \approx k \frac{M}{r}, \quad k = 0, 1, \dots, r-1 \quad (8.5.1)$$

r peaks of equal height

$$\frac{y}{M} \approx \frac{k}{r} \quad (8.5.2)$$

what one measurement tells you

If r divides M the peaks are exact integers. Otherwise each peak is spread over neighbouring values, but the nearest integer still carries a large share of its weight.

$$P\left(\left|\frac{y}{M} - \frac{k}{r}\right| \leq \frac{1}{2M}\right) \geq \frac{4}{\pi^2 r} \text{ for each } k \quad (8.5.3)$$

the nearest integer to each peak: in total at least $4/\pi^2 \approx 0.41$

MEASURED y FOR $N = 15, a = 7, M = 256$	y/M	FRACTION k/r	CANDIDATE r
0	0	0/4	none: try again
64	0.25	1/4	4 ✓
128	0.5	2/4 = 1/2	2 ✗ ($7^2 \not\equiv 1$)
192	0.75	3/4	4 ✓

The table shows the two ways a run can fail. $k = 0$ carries no information, and when k shares a factor with r the fraction reduces and reveals only a divisor of r . Both are caught by checking whether $a^r \equiv 1$, and both are fixed by running again or by combining candidates.

$$r = \text{lcm}(r_1, r_2) \quad \text{from two runs giving reduced denominators } r_1, r_2 \quad (8.5.4)$$

combining partial answers

WORKED EXAMPLE · WHY $r = 4$ GIVES EXACT PEAKS

$M = 256 = 4 \cdot 64$, so $M/r = 64$ is an integer and $y \in \{0, 64, 128, 192\}$ each with probability exactly $\frac{1}{4}$.

Half of all runs ($k = 1$ or 3) give $r = 4$ immediately.

LESSON 8.6

Continued fractions

KEY IDEA

y/M is a rational number close to k/r , with $r < N$. The continued-fraction expansion of y/M finds the best such approximation efficiently, and a classic theorem says it will be among them.

$$\frac{y}{M} = a_0 + \frac{1}{a_1 + \frac{1}{a_2 + \frac{1}{\ddots}}} = [a_0; a_1, a_2, \dots] \quad (8.6.1)$$

the continued-fraction expansion

$$p_j = a_j p_{j-1} + p_{j-2}, \quad q_j = a_j q_{j-1} + q_{j-2}, \quad \frac{p_j}{q_j} = [a_0; a_1, \dots, a_j] \quad (8.6.2)$$

convergents, from $p_{-1} = 1, p_{-2} = 0, q_{-1} = 0, q_{-2} = 1$

$$\left| \frac{y}{M} - \frac{k}{r} \right| \leq \frac{1}{2r^2} \Rightarrow \frac{k}{r} \text{ is a convergent of } \frac{y}{M} \quad (8.6.3)$$

Legendre's theorem

The measurement guarantees an error of at most $1/(2M)$. Choosing the counting register large enough, $M \geq N^2$, makes that at most $1/(2N^2) < 1/(2r^2)$, so the theorem applies. This is why $t \approx 2n$.

$$M = 2^t \geq N^2 \quad (8.6.4)$$

the size of the counting register

The recipe: expand y/M , and for each convergent p_j/q_j with $q_j < N$, test whether $a^{q_j} \equiv 1 \pmod{N}$. The first that passes is a candidate order. The expansion has $O(n)$ terms, so this is fast.

WORKED EXAMPLE · $N = 21$, $A = 2$, $M = 512$

The order is $r = 6$, since $2^6 = 64 = 3 \cdot 21 + 1$. A run returns $y = 427$, the nearest integer to $5 \cdot 512/6 \approx 426.7$.

$427/512 = [0; 1, 5, 42, 2]$: $512 = 1 \cdot 427 + 85$, $427 = 5 \cdot 85 + 2$, $85 = 42 \cdot 2 + 1$, $2 = 2 \cdot 1$.

Convergents: $0/1, 1/1, 5/6, 211/253, 427/512$. The first denominator that works is 6:
 $2^6 \equiv 1 \pmod{21}$.

Then $2^3 = 8 \not\equiv -1$, so $\gcd(7, 21) = 7$ and $\gcd(9, 21) = 3$: $21 = 3 \times 7$.

LESSON 8.7

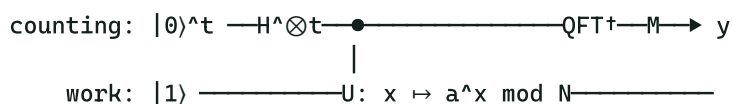
The whole algorithm

KEY IDEA

Shor's algorithm is a classical reduction wrapped around one quantum subroutine, period finding. Everything before and after the quantum step is ordinary number theory.

STEP	WHAT HAPPENS	WHERE
1	if N is even or a prime power, factor it directly	classical
2	pick random a ; if $\gcd(a, N) > 1$, that gcd is a factor	classical
3	prepare $ 0\rangle^{\otimes t} 1\rangle$, apply $H^{\otimes t}$	quantum
4	controlled modular exponentiation, $ x\rangle 1\rangle \mapsto x\rangle a^x \bmod N\rangle$	quantum
5	apply $\text{QFT}^\dagger$ to the counting register and measure y	quantum
6	continued fractions on y/M , giving a candidate r ; check $a^r \equiv 1$	classical
7	if r is even and $a^{r/2} \not\equiv -1$, output $\gcd(a^{r/2} \pm 1, N)$; otherwise repeat	classical

PERIOD FINDING



Each run succeeds with a probability bounded below by a slowly shrinking function of N , at worst proportional to $1/\log \log N$, so a small number of repetitions finds the factors.

$$\text{qubits} \approx \underbrace{2n}_{\text{counting}} + \underbrace{n}_{\text{work}} + O(n)_{\text{ancillas}}, \quad \text{gates} = O(n^3) \quad (8.7.1)$$

the size of one run

Careful circuit design shrinks the width. The counting register can be replaced by a single qubit that is measured and reset $2n$ times, with the phase corrections applied classically. That gives constructions with $2n + 3$ qubits in total (Beauregard, 2003).

$$\text{semiclassical QFT: } 2n \text{ counting qubits} \rightarrow 1 \text{ recycled qubit} \quad (8.7.2)$$

the width saving, paid for in mid-circuit measurements

WORKED EXAMPLE · COUNTING FOR RSA-2048

$n = 2048$: about 4096 counting qubits and 2048 work qubits in the textbook layout, or around 4100 logical qubits in a $2n + 3$ design.

Every one of those must be a *logical*, error-corrected qubit, which is where lesson 8.8 begins.

LESSON 8.8

Error budgets

KEY IDEA

A circuit of G gates with error p each succeeds with probability about e^{-pG} . Shor's circuits are millions to billions of gates, so they need error correction, and error correction multiplies the qubit count by roughly a thousand.

$$P_{\text{success}} \approx (1 - p)^G \approx e^{-pG} \quad (8.8.1)$$

unprotected gates

With today's physical error rates near 10^{-3} , a million-gate circuit has $pG = 1000$ and essentially no chance of running cleanly. Only circuits of a few hundred gates survive without correction.

$$p_L \approx A \left(\frac{p}{p_{\text{th}}} \right)^{\lfloor (d+1)/2 \rfloor}, \quad n_{\text{phys}} \approx 2d^2 \quad (8.8.2)$$

surface code of distance d : logical error rate and physical qubits per logical qubit

Below the threshold $p_{\text{th}} \approx 10^{-2}$, every increase of the code distance by two multiplies the logical error rate by p/p_{th} . The price is qubits, growing as d^2 .

WORKED EXAMPLE · SIZING ONE LOGICAL QUBIT

Take $p = 10^{-3}$, $p_{\text{th}} = 10^{-2}$, $A = 0.1$, and a target of $p_L \approx 10^{-12}$ for a long computation.

Then $0.1 \times 0.1^{(d+1)/2} \leq 10^{-12}$ requires $(d+1)/2 \geq 11$, so $d = 21$ and $n_{\text{phys}} \approx 2 \cdot 21^2 = 882$: roughly a thousand physical qubits per logical qubit.

ESTIMATE FOR RSA-2048	PHYSICAL QUBITS	RUN TIME
Gidney and Ekerå, 2019	about 20 million	about 8 hours
Gidney, 2025	under 1 million	under a week

Against that scale, factoring 15 or 21 is a demonstration of the circuit's parts, not of its power. Several early demonstrations also used circuits simplified with knowledge of the answer, which Smolin, Smith and Vargo (2013) showed can “factor” numbers of any size without doing any real work.

Estimates assume a physical error rate near 10^{-3} and nearest-neighbour connectivity, and they have fallen steadily as algorithms and error-correction schemes improve. Treat them as orders of magnitude.

LESSON 8.9

What this means for cryptography

KEY IDEA

Shor breaks every widely deployed public-key scheme built on factoring or discrete logarithms. Symmetric cryptography only needs larger keys. The replacement schemes are standardised, and migration has begun, because encrypted data can be stored today and decrypted later.

SCHEME	HARD PROBLEM	QUANTUM ATTACK	STATUS
RSA	factoring	Shor	broken
Diffie–Hellman, DSA	discrete logarithm	Shor	broken
elliptic-curve (ECDH, ECDSA)	elliptic-curve discrete log	Shor	broken
AES, ChaCha20	key search	Grover	use 256-bit keys
SHA-2, SHA-3	preimages	Grover	use longer outputs
ML-KEM, ML-DSA	module lattices	none known	standardised 2024
SLH-DSA	hash functions	Grover only	standardised 2024

$$\text{AES-}k : 2^k \rightarrow 2^{k/2}, \quad \text{hash preimage: } 2^h \rightarrow 2^{h/2}$$

Grover's effect on symmetric primitives (module 7)

(8.9.1)

Elliptic-curve cryptography falls to a variant of Shor's algorithm for discrete logarithms, and it needs fewer logical qubits than RSA at comparable classical security, because its keys are much shorter.

$$\text{ECDLP on an } n\text{-bit curve: } \leq 9n + 2\lceil \log_2 n \rceil + 10 \text{ logical qubits} \quad (n = 256 : 2330)$$

an upper-bound estimate (Roetteler et al., 2017)

(8.9.2)

In August 2024 NIST published its first post-quantum standards: FIPS 203 (ML-KEM, from Kyber) for key exchange, and FIPS 204 (ML-DSA, from Dilithium) and FIPS 205 (SLH-DSA, from SPHINCS+) for signatures. HQC was selected in 2025 as a backup key-encapsulation scheme based on different mathematics.

DEFINITION · HARVEST NOW, DECRYPT LATER

An adversary records encrypted traffic today and decrypts it once a large quantum computer exists. Data that must stay secret for years is therefore already at risk, even though no such computer exists yet.

$$x + y > z \Rightarrow \text{start migrating now}$$

(8.9.3)

Mosca's inequality: x years of required secrecy, y years to migrate, z years until a cryptographically relevant quantum computer

WORKED EXAMPLE · APPLYING MOSCA'S INEQUALITY

Medical records must stay confidential for $x = 25$ years, and migrating a hospital network's systems takes $y = 5$.

If a cryptographically relevant quantum computer is $z = 15$ years away, then $x + y = 30 > 15$. The last records encrypted with RSA, at the end of the migration, would still need $x + y - z = 15$ years of secrecy after that computer arrives, so the migration is already late.

MODULE 8 · FORMULA SHEET

Formula sheet

The key results of the module, numbered as they appear in the lessons.

an RSA key: public (N, e) , private d	$N = pq, \quad \varphi(N) = (p-1)(q-1), \quad ed \equiv 1 \pmod{\varphi(N)}$	(8.1.1)
encryption and decryption	$c = m^e \bmod N, \quad m = c^d \bmod N$	(8.1.2)
polynomial, with schoolbook arithmetic: faster multiplication lowers it further	Shor: $O(n^3)$ gates for an n -bit N	(8.1.4)
the order of a modulo N , defined when $\gcd(a, N) = 1$	$r = \min \{ r > 0 : a^r \equiv 1 \pmod{N} \}$	(8.2.1)
the factors, when r is even and $a^{r/2} \not\equiv -1 \pmod{N}$	$\gcd(a^{r/2} - 1, N) \quad \text{and} \quad \gcd(a^{r/2} + 1, N)$	(8.2.2)
modular multiplication: a permutation, so a unitary, because $\gcd(a, N) = 1$	$U_a y\rangle = ay \bmod N\rangle \quad (0 \leq y < N)$	(8.3.1)
controlled powers: one controlled $U_{a^{2^j}}$ per bit x_j of the counting register	$ x\rangle 1\rangle \mapsto x\rangle a^x \bmod N\rangle, \quad a^x = \prod_{j=0}^{t-1} (a^{2^j})^{x_j}$	(8.3.2)
eigenstates of modular multiplication	$ u_s\rangle = \frac{1}{\sqrt{r}} \sum_{k=0}^{r-1} e^{-2\pi i s k / r} a^k \bmod N\rangle, \quad U_a u_s\rangle = e^{2\pi i s / r} u_s\rangle$	(8.3.3)
the quantum Fourier transform	$\text{QFT} x\rangle = \frac{1}{\sqrt{M}} \sum_{y=0}^{M-1} e^{2\pi i x y / M} y\rangle, \quad M = 2^t$	(8.4.1)
the controlled rotations	$R_k = \begin{pmatrix} 1 & 0 \\ 0 & e^{2\pi i / 2^k} \end{pmatrix}, \quad R_1 = Z, \quad R_2 = S, \quad R_3 = T$	(8.4.2)
phase estimation: the phase, written out as a binary number	$\frac{1}{\sqrt{M}} \sum_{x=0}^{M-1} e^{2\pi i \varphi x} x\rangle \xrightarrow{\text{QFT}^\dagger} \approx \widetilde{M\varphi}\rangle$	(8.4.4)
r peaks of equal height	$P(y) \approx \frac{1}{r} \text{ for } y \approx k \frac{M}{r}, \quad k = 0, 1, \dots, r-1$	(8.5.1)
what one measurement tells you	$\frac{y}{M} \approx \frac{k}{r}$	(8.5.2)
the continued-fraction expansion	$\frac{y}{M} = a_0 + \frac{1}{a_1 + \frac{1}{a_2 + \frac{1}{\ddots}}}$	(8.6.1)
convergents, from $p_{-1} = 1, p_{-2} = 0, q_{-1} = 0, q_{-2} = 1$	$p_j = a_j p_{j-1} + p_{j-2}, \quad q_j = a_j q_{j-1} + q_{j-2}, \quad \frac{p_j}{q_j} = [a_0; a_1, \dots, a_j]$	(8.6.2)
Legendre's theorem	$\left \frac{y}{M} - \frac{k}{r} \right \leq \frac{1}{2r^2} \Rightarrow \frac{k}{r} \text{ is a convergent of } \frac{y}{M}$	(8.6.3)
the size of the counting register	$M = 2^t \geq N^2$	(8.6.4)

the size of one run	$\text{qubits} \approx \underbrace{2n}_{\text{counting}} + \underbrace{n}_{\text{work}} + O(n)_{\text{ancillas}}, \quad \text{gates} = O(n^3)$	(8.7.1)
---------------------	--	---------

unprotected gates	$P_{\text{success}} \approx (1 - p)^G \approx e^{-pG}$	(8.8.1)
-------------------	--	---------

surface code of distance d : logical error rate and physical qubits per logical qubit	$p_L \approx A \left(\frac{p}{p_{\text{th}}} \right)^{\lfloor (d+1)/2 \rfloor}, \quad n_{\text{phys}} \approx 2d^2$	(8.8.2)
---	--	---------

Grover's effect on symmetric primitives (module 7)	$\text{AES-}k : 2^k \rightarrow 2^{k/2}, \quad \text{hash preimage: } 2^h \rightarrow 2^{h/2}$	(8.9.1)
--	--	---------

Mosca's inequality: x years of required secrecy, y years to migrate, z years until a cryptographically relevant quantum computer	$x + y > z \Rightarrow \text{start migrating now}$	(8.9.3)
--	--	---------
